INDIAN MONOPOLY

Online Multiplayer Game
Coding-Agent Implementation Manual

A production-minded blueprint for a GitHub-hosted web application with Firebase backend, Google authentication, friends, private rooms, real-time multiplayer gameplay, reactions, and post-game session archiving.

PROJECT DECISION	RECOMMENDED BASELINE
Frontend hosting	GitHub repository + GitHub Pages (static frontend build)
Frontend	React + TypeScript + Vite
Backend	Firebase Authentication + Cloud Firestore + Cloud Functions
Live game transport	Firestore real-time listeners for MVP; presence/ephemeral reactions via Realtime Database where useful
State authority	Server-authoritative game engine in Cloud Functions
CI/CD	GitHub Actions
Target	Responsive desktop, tablet and mobile browsers

Version 1.0 | 3 November 2026 | Prepared as an implementation specification for an AI coding agent

How the coding agent must use this manual

Treat this document as the source of truth for product scope, architecture, game rules, data contracts, security boundaries, and acceptance criteria. Do not generate the whole application in one uncontrolled pass. Work in milestones, keep the app runnable after each milestone, write tests before connecting UI to live game state, and ask for clarification only when a decision materially blocks implementation.

Non-negotiable principle: the browser is an untrusted client. It may render the board and request actions, but it must never decide authoritative dice outcomes, rent, ownership, payments, turn order, auction winners, or victory. Those must be validated and committed by trusted backend code.

Document map

- 1. Product vision and scope
- 2. Functional requirements and user journeys
- 3. Technology stack and deployment architecture
- 4. Repository structure and engineering standards
- 5. Game rules and authoritative game engine
- 6. Firebase data model and lifecycle
- 7. Real-time multiplayer, presence and reconnection
- 8. Google login, profiles and friends
- 9. Security rules and backend trust boundary
- 10. UI/UX system and accessibility
- 11. Testing, observability and quality gates
- 12. CI/CD, hosting and environment setup
- 13. Privacy, retention and operational safeguards
- 14. Delivery milestones and agent workflow
- 15. Acceptance criteria and launch checklist
- Appendices: example schemas, callable APIs and test scenarios

1. Product vision and scope

1.1 Product statement

Build a minimalist, responsive, browser-based Indian property-trading board game. Users sign in with Google, add friends, create or join private rooms, invite friends using a room code/link, and play synchronously from separate devices. Game state updates in real time. Players can react with a small curated set of emotes. There is no chat or direct messaging in this release.

1.2 Primary goals

- Low-friction onboarding: Google sign-in, profile creation and a clear first action.
- Room-based multiplayer: host creates a room, chooses settings, shares invite code, and starts when enough players have joined.
- Consistent gameplay: every player sees the same authoritative state and turn progression.
- Simple controls: obvious dice, legal actions, clear rent/payment explanations and short animations.
- Reliable reconnect: temporary network loss should not destroy a player's seat or game.
- Session archive: completed games are retained as historical records; live room/session documents and ephemeral state are cleaned up.
- Mobile-first usability without sacrificing a clear desktop board.

1.3 Explicitly out of scope for MVP

- Text chat, voice chat, video, user-to-user messaging and public global matchmaking.
- Real-money payments, purchases, betting, prizes or cash-out.
- Spectator mode, tournaments, ranked matchmaking and public leaderboards.
- Player-created custom cards or custom rule scripting.
- Native iOS/Android apps; the first release is a responsive web app/PWA-ready site.
- Complex dynamic stock-market mechanics or real Indian financial products.

1.4 Product naming and IP caution

Use a distinct project name and original artwork, UI, card text and board presentation. “Monopoly” is a protected commercial brand and game identity. Do not use Hasbro logos, exact board artwork, distinctive branded typography, official card wording, or imply affiliation. The internal working title “Indian Monopoly” should be reviewed before public release; a safer public-facing original name is recommended.

1.5 MVP game configuration

Setting	MVP value
Players per room	2–6
Starting cash	■15,000 per player
Board	40 spaces; 22 city properties, 4 transit, 2 utilities, tax and event spaces
Start reward	■2,000
Dice	Two six-sided dice, server-generated
Room type	Private, invite-code based
Game modes	Standard game; optional 90-minute net-worth mode after core testing
Emotes	Curated reaction set; no arbitrary text
Game archive	Completed game summary + final standings + event history
Live state retention	Delete after archive succeeds and configured grace period expires

2. Functional requirements and user journeys

2.1 First-time user

- User opens the GitHub Pages URL and sees a lightweight landing page with “Sign in with Google”.
- Firebase Authentication completes Google OAuth using the approved domain configuration.
- On first sign-in, create a minimal user profile with immutable Firebase UID, display name, avatar URL, createdAt and updatedAt.
- Show the home dashboard: Create Room, Join Room, Friends, Recent Games and Settings.
- Do not ask for phone number, date of birth, address or unrelated personal data.

2.2 Create room

- Authenticated user selects Create Room.
- Client requests a room from a callable Cloud Function.
- Backend validates user eligibility and creates a cryptographically random, human-readable invite code (avoid ambiguous characters such as O/0 and I/1).
- Host becomes first player and room owner. Store only a normalized/hashable code lookup where practical; do not expose room enumeration.
- Host can choose allowed settings: max players (2–6), turn timer (off/30/45/60 seconds), game mode and optional bots only if implemented later.
- Room lobby displays joined players, ready state, host marker, invite code, copy invite link and start eligibility.

2.3 Join room

- User enters invite code or opens a deep link.
- Client calls joinRoom; backend validates code, room status, capacity, bans/blocks if implemented, and duplicate membership.
- Backend atomically adds membership and player seat. Repeated requests by the same user must be idempotent.
- Lobby updates for all members through a Firestore listener.
- Do not allow joining after the game has started.

2.4 Ready and start

- Each non-host player toggles Ready. Host may also ready.
- Start is enabled only when at least two players have joined and all joined players are ready.
- Host requests startGame; backend revalidates all preconditions and creates the canonical initial game state.
- Server assigns player order deterministically (e.g. joinedAt order, or a server-generated shuffled order if product decides this).
- After start, room membership is frozen; leaving does not silently remove a player from the board.

2.5 In-game player journey

- All players see the same board, tokens, active player, dice result, transaction log and property ownership.
- Only the active player sees enabled turn actions. Other players see a waiting state and may send emotes.
- Player presses Roll. The client submits a command; backend validates turn and phase, generates dice, applies movement and destination resolution, then commits state.
- Where a choice is required (buy/decline, auction bid, build, mortgage, jail decision), backend records a pending action and only valid actions are enabled.
- Every action returns a server-confirmed result and is broadcast to room members.
- Use optimistic UI only for non-authoritative presentation such as button press feedback; never optimistically mutate money or ownership.

2.6 Finish and archive

- When the game reaches a terminal state, backend marks it completed and writes an immutable archive record.
- Archive contains final standings, player IDs/display-name snapshots, final cash/net worth, winner, ruleset version, start/end timestamps and a bounded event summary.
- Only after archive write is confirmed should cleanup begin.
- Delete live game state, room state, presence, pending actions, ephemeral reactions and invite-code lookup after a short grace period.
- Retain archive according to the stated retention policy. A player can view completed games they participated in, subject to access rules.

3. Technology stack and architecture

3.1 Recommended stack

Layer	Technology	Reason / usage
Language	TypeScript (strict)	Shared domain types; safer client/server contracts.
Frontend	React + Vite	Fast static build suitable for GitHub Pages.
Routing	React Router	Home, lobby, game, friends, profile and history routes.
Styling	Tailwind CSS + CSS variables	Consistent responsive design tokens; avoid heavy UI kits.
UI primitives	Radix UI or shadcn/ui (selectively)	Accessible dialogs, menus, tooltips and popovers.
State	TanStack Query + lightweight Zustand	Server data/cache vs local transient UI state.
Forms	React Hook Form + Zod	Validated settings, join code and profile forms.
Backend	Firebase Cloud Functions (TypeScript)	Trusted game commands, room lifecycle, cleanup and archive.
Auth	Firebase Authentication	Google sign-in and identity.
Persistent database	Cloud Firestore	Profiles, friends, rooms, live game state and archives.
Presence	Firebase Realtime Database	Online/offline presence and transient connection state.
Scheduled jobs	Cloud Functions scheduled functions	Expiry and cleanup of abandoned rooms/sessions.
Hosting	GitHub Pages	Static frontend deployment from GitHub Actions.
CI/CD	GitHub Actions	Lint, typecheck, tests, build and deploy.
Testing	Vitest + Testing Library + Playwright	Unit, component and end-to-end coverage.
Monitoring	Firebase/Google Cloud logs + Crashlytics alternative for web	Backend errors, performance and client error telemetry.
Code quality	ESLint + Prettier + TypeScript	Consistent, enforceable standards.

Important hosting note: GitHub Pages serves static files; it is not the application backend and cannot run trusted server code. Firebase provides the backend. Use a Vite build with a GitHub Pages base path, and configure SPA route fallback (or use hash routing) so deep links do not 404. A custom domain is preferable for OAuth configuration and a cleaner invite URL.

3.2 High-level architecture

```
PLAYER BROWSER (React + TypeScript, hosted on GitHub Pages)
   | Firebase Auth: Google OAuth
   | Callable HTTPS functions: commands only
   | Firestore listeners: room/game read updates
   | Realtime Database: presence + transient reactions
   v
FIREBASE PROJECT
   ■■■ Authentication
   ■■■ Cloud Firestore
   ■    ■■■ users
   ■    ■■■ friendRequests / friendships
   ■    ■■■ rooms
   ■    ■■■ liveGames
   ■    ■■■ gameArchives
   ■■■ Cloud Functions (trusted domain layer)
   ■    ■■■ createRoom / joinRoom / leaveRoom
   ■    ■■■ readyPlayer / startGame
   ■    ■■■ submitGameAction
   ■    ■■■ sendReaction
   ■    ■■■ finishAndArchiveGame
   ■    ■■■ scheduled cleanup
   ■■■ Realtime Database
        ■■■ presence
        ■■■ roomReactions (TTL/short-lived)

GITHUB REPOSITORY
   ■■■ source code
   ■■■ GitHub Actions CI
   ■■■ deploy static build to GitHub Pages
```

3.3 Firestore vs Realtime Database

Use Firestore for durable, queryable application records and authoritative game snapshots. Use Realtime Database for presence because its connection lifecycle and onDisconnect support are useful for online status. Reactions can use RTDB with a short TTL, or Firestore if a simple auditable event feed is preferred. Do not duplicate the entire game state in both databases.

3.4 Why server-authoritative commands

A client-only multiplayer game is vulnerable to modified browser code, forged dice, fake rent, replayed actions and conflicting writes. The Cloud Function must validate the authenticated UID, room membership, active player, turn phase, action payload and current version before applying an action. Use a Firestore transaction for state transition and append the corresponding event in the same transaction where possible.

3.5 Region and cost

Choose one Firestore/Functions region close to the expected player base (for India, evaluate an available India region and keep Firestore and Functions co-located). Confirm current product availability, quotas, billing requirements and pricing before selecting. Cloud Functions deployment and scheduled functions may require the Blaze billing plan. Set budget alerts, quotas where supported, and abuse controls before public launch.

4. Repository structure and engineering standards

```
indian-board-game/
├── .github/
│   ├── workflows/
│       ├── ci.yml
│       ├── deploy-pages.yml
├── public/
│   ├── favicon.svg
│   ├── assets/
├── src/
│   ├── app/
│   │   ├── App.tsx
│   │   ├── router.tsx
│   │   ├── providers/
│   ├── components/
│   │   ├── ui/
│   │   ├── board/
│   │   ├── player/
│   │   ├── feedback/
│   ├── features/
│   │   ├── auth/
│   │   ├── dashboard/
│   │   ├── rooms/
│   │   ├── friends/
│   │   ├── game/
│   │   ├── history/
│   │   ├── settings/
│   ├── lib/
│   │   ├── firebase/
│   │   ├── api/
│   │   ├── validation/
│   │   ├── utils/
│   ├── domain/
│   │   ├── game-types.ts
│   │   ├── board-config.ts
│   │   ├── ruleset.ts
│   │   ├── action-schemas.ts
│   ├── hooks/
│   ├── styles/
│   ├── main.tsx
├── functions/
│   ├── src/
│       ├── index.ts
│       ├── callable/
│       ├── game-engine/
│       ├── room-service/
│       ├── friend-service/
│       ├── archive-service/
│       ├── cleanup/
│       ├── shared/
│       ├── tests/
├── firestore.rules
├── firestore.indexes.json
├── database.rules.json
├── firebase.json
├── .firebaserc.example
├── .env.example
├── package.json
├── tsconfig.json
├── vite.config.ts
├── README.md
├── docs/
    ├── architecture.md
    ├── rules.md
    ├── runbooks.md
```

4.1 Standards

- TypeScript strict mode enabled; avoid any. Use unknown and schema parsing at trust boundaries.
- Use Zod schemas for callable request payloads and shared domain validation.
- Keep pure game rules independent of Firebase so they can be unit-tested deterministically.
- Use dependency injection for random number generation, clock and persistence adapters in tests.

- Use integer rupees only; never floating-point money. If paise are introduced, store integer paise.
- Use UTC server timestamps for persistence; render local time in UI.
- Every command includes commandId/idempotency key and expected game version.
- Never store secrets, service-account credentials or private keys in the frontend or Git repository.
- Keep board configuration and ruleset versioned. Historical archives must record rulesetVersion.
- Use conventional commits and small pull requests/commits per milestone.

5. Game rules and authoritative engine

5.1 Ruleset baseline

Use the previous Indian-themed board specification as the starting ruleset, but treat this section as the authoritative implementation baseline. The coding agent must put all prices, rents, card instructions and special-space behaviour in configuration rather than scattering constants through components.

Rule	Baseline
Board spaces	40, clockwise, positions 0–39
Players	2–6
Starting cash	■15,000
Start reward	■2,000 when passing or landing on Start clockwise
Dice	Two server-generated d6
Doubles	Extra roll; three consecutive doubles in one turn sends player to Jail
Jail bail	■500; up to three doubles attempts
GST Checkpost	■2,000
Infrastructure Levy	■1,000
Property groups	8 colour groups; 22 properties
Transit	4 spaces; rent depends on active transit assets owned
Utilities	2 spaces; dice multiplier rent
Development	Even building; 4 blocks then tower
Mortgage	50% of price; redemption at 110% of mortgage value
Victory	Last solvent player; optional 90-minute net-worth mode

The previous draft's illustrative property/rent figures should be stored as a versioned configuration file. Before release, run balance simulations: the sample tower rent multipliers are intentionally aggressive and may cause early elimination. Keep them configurable and tune from playtests.

5.2 Game state machine

```
LOBBY
  ■■ host starts (2..6 players, all ready) -> INITIALIZING
INITIALIZING
  ■■ server creates board state and turn order -> IN_PROGRESS
IN_PROGRESS
  ■■ normal action -> IN_PROGRESS
  ■■ pending purchase/auction/jail choice -> ACTION_REQUIRED
  ■■ player bankruptcy -> IN_PROGRESS (if >1 solvent player)
  ■■ one solvent player / time limit -> COMPLETED
ACTION_REQUIRED
  ■■ valid response -> IN_PROGRESS
  ■■ timeout policy -> IN_PROGRESS
COMPLETED
  ■■ archive committed -> ARCHIVED
ARCHIVED
  ■■ live data cleanup -> DELETED
```

5.3 Turn phases and allowed actions

Phase	Allowed command	Server checks
AWAIT_ROLL	ROLL_DICE	Caller is current player; no unresolved action; player active.
JAIL_DECISION	PAY_BAIL / TRY_DOUBLES / USE_JAIL_CARD	Player jailed; legal attempt count; sufficient cash/card.
PROPERTY_DECISION	BUY_PROPERTY / DECLINE_PROPERTY	Landing property is unowned; player can afford buy.
AUCTION	PLACE_BID / PASS_BID	Auction open; bidder active; bid increment and cash valid.
MANAGEMENT	BUILD / SELL_BUILDING / MORTGAGE / UNMORTGAGE / PROPOSE_TRADE	Ownership, even-build, inventory and cash constraints.
TRADE_RESPONSE	ACCEPT_TRADE / REJECT_TRADE	Offer exists, parties active, deeds transferable.
TURN_END	END_TURN	No mandatory pending action; active player and phase valid.

5.4 Critical rule-engine decisions

- Dice are generated only on the server using a cryptographically secure random source where available. Never accept dice values from the client.
- Use a single canonical board array with unique space IDs and explicit space type/configuration.
- Resolve card movement recursively but guard against loops; impose a maximum nested-resolution depth.
- Movement crossing Start pays exactly once for clockwise movement; direct jail movement does not pass Start.
- Do not let an action proceed if the game version has changed since the client rendered it.
- Every payment checks available cash and invokes a consistent debt/liquidation flow.
- Bankruptcy transfers deeds/cards according to creditor type and ruleset; no orphaned ownership.
- Turn timers are server timestamps, not browser timers. Browser countdown is display-only.
- Define deterministic timeout policy: auto-pass optional decisions; for mandatory payments, initiate debt handling; for auction, pass when timer expires.
- Use an append-only event log for meaningful state changes; keep a compact current snapshot for fast reads.

5.5 Game action envelope

```
type GameCommand = {
  commandId: string;       // UUID generated once per user action
  roomId: string;
  gameId: string;
  expectedVersion: number;
  type: GameActionType;
  payload: Record<string, unknown>;
};

type GameActionType =
  | "ROLL_DICE"
  | "BUY_PROPERTY"
  | "DECLINE_PROPERTY"
  | "PLACE_BID"
  | "PASS_BID"
  | "PAY_BAIL"
  | "TRY_DOUBLES"
  | "USE_JAIL_CARD"
  | "BUILD"
  | "SELL_BUILDING"
  | "MORTGAGE"
  | "UNMORTGAGE"
  | "PROPOSE_TRADE"
  | "ACCEPT_TRADE"
  | "REJECT_TRADE"
  | "END_TURN";
```

5.6 Idempotency and concurrency

Each command must be idempotent. Store processed command IDs (or a bounded command receipt subcollection) so retrying after a mobile network timeout does not charge rent twice or roll dice twice. Use a Firestore transaction that reads the current game version, validates expectedVersion, applies one transition, increments version, and writes the event/receipt atomically. On version conflict, return a typed conflict response; the client refreshes state and explains that the board changed.

5.7 Randomness and fairness

The server is authoritative for dice and card shuffles. Store the resulting dice values and card IDs in the event history. Do not store a reusable random seed in client-readable documents. For stronger auditability, consider a commit/reveal scheme in a later release; it is not required for MVP.

6. Firebase data model and lifecycle

6.1 Firestore collections

Path	Purpose	Access
users/{uid}	Public profile fields and account metadata	Owner read/write allowlist; server controls sensitive fields
friendRequests/{requestId}	Pending friend request	Sender/recipient scoped; validate via functions
friendships/{pairId}	Accepted relationship	Only participants read; server writes
rooms/{roomId}	Lobby metadata, host, status, invite-code lookup metadata	Members read; server writes
rooms/{roomId}/members/{uid}	Room seat, ready state, joinedAt	Members read; ready change via function
games/{gameId}	Live authoritative game snapshot	Members read; server writes only
games/{gameId}/events/{eventId}	Append-only action/event history	Members read; server writes only
gameArchives/{archiveId}	Completed game history	Participants read; server writes only
commandReceipts/{receiptId}	Idempotency and command result	Server only

6.2 Suggested user document

```
users/{uid}
{
  uid: string,
  displayName: string,
  photoURL: string | null,
  createdAt: Timestamp,
  updatedAt: Timestamp,
  friendCount: number,       // denormalized, server-maintained
  privacy: {
    allowFriendRequests: "everyone" | "friendsOfFriends" | "nobody"
  }
}
```

Do not duplicate Google access tokens or OAuth credentials into Firestore. Use Firebase Auth identity. Avoid storing email unless the product genuinely needs it; if stored, treat it as private and expose only to the account owner or trusted backend.

6.3 Room document

```
rooms/{roomId}
{
  roomId: string,
  inviteCodeHash: string,
  hostUid: string,
  status: "LOBBY" | "IN_PROGRESS" | "COMPLETED" | "ARCHIVED",
  maxPlayers: number,
  gameMode: "STANDARD" | "TIMED",
  turnTimerSeconds: 0 | 30 | 45 | 60,
  memberUids: string[],      // small bounded array (max 6)
  createdAt: Timestamp,
  updatedAt: Timestamp,
  expiresAt: Timestamp
}
```

6.4 Live game document

```
games/{gameId}
{
  roomId: string,
  status: "IN_PROGRESS" | "ACTION_REQUIRED" | "COMPLETED",
  rulesetVersion: "1.0.0",
  version: number,
  playerOrder: string[],
  currentPlayerUid: string,
  turnNumber: number,
  phase: string,
  board: { properties: PropertyState[], spaces: SpaceState[] },
  players: PlayerState[],
  pendingAction: PendingAction | null,
  dice: { first: number, second: number, rolledAt: Timestamp } | null,
  startedAt: Timestamp,
  updatedAt: Timestamp,
  completedAt: Timestamp | null
}
```

Keep the snapshot bounded. A 40-space board and six players are small, but do not put unbounded event history into the same document. Firestore document size is limited; store events in a subcollection.

6.5 Event document

```
games/{gameId}/events/{eventId}
{
  sequence: number,
  commandId: string,
  actorUid: string,
  type: string,
  payload: object,          // validated, minimal, no secrets
  createdAt: Timestamp,
  versionAfter: number
}
```

6.6 Completed archive

```
gameArchives/{archiveId}
{
  gameId: string,
  roomId: string,
  rulesetVersion: string,
  playerSnapshots: [
    { uid, displayName, photoURL, finalCash, netWorth, rank }
  ],
  winnerUid: string,
  startedAt: Timestamp,
  completedAt: Timestamp,
  durationSeconds: number,
  eventSummary: object[],   // bounded summary, not necessarily every event
  archiveVersion: 1,
  participantUids: string[],
  retentionUntil: Timestamp
}
```

6.7 Session cleanup design

The user requirement is: completed games should be saved, while the live session and associated transient information should be removed. Implement this as a two-stage archival workflow, not a single delete.

- Stage 1 — Complete: transaction marks game COMPLETED, stops new commands, calculates final standings and creates an archive job/outbox record.
- Stage 2 — Archive: trusted function copies a verified final snapshot and required summary into gameArchives. Use deterministic archive ID (e.g. gameId) to make retries safe.
- Stage 3 — Verify: check archive exists, required fields are present, and participantUids are correct.
- Stage 4 — Cleanup: delete live game, room, members, event documents, command receipts, invite lookup and ephemeral reactions. Use recursive deletion for subcollections; deleting a Firestore parent document does not automatically delete its subcollections.
- Stage 5 — Retention: preserve archive until retentionUntil; apply a scheduled retention policy or TTL where appropriate. TTL is asynchronous and should not be treated as immediate deletion.
- Abandoned lobby: expire after configurable inactivity (e.g. 24 hours); delete room and members.
- Abandoned active game: do not auto-delete immediately. Mark stale, notify/reconnect participants, and apply a clearly documented abandonment policy (e.g. host resume or expire after a longer period).

Use an outbox/cleanup job so a transient function failure cannot cause the only copy of a completed game to be deleted. Archive first, verify, then delete. Keep an operational log of cleanup failures without retaining unnecessary player data.

7. Real-time multiplayer, presence and reconnection

7.1 State delivery

Clients subscribe to the room and live game snapshot using Firestore onSnapshot listeners. The backend writes one canonical state transition per accepted command. All clients render from that state. Keep the game snapshot compact; use event history for the activity feed.

7.2 Presence

Use Realtime Database presence per authenticated UID, with onDisconnect to mark the connection offline. A user's presence should not determine ownership or cause automatic forfeiture. In a room, display “Online”, “Reconnecting” or “Offline” based on last-seen timestamp and active connection state.

7.3 Reconnection

- On reconnect, re-authenticate, rejoin listeners and fetch the latest game snapshot.
- Never replay a command automatically unless the client knows its commandId and the server can return the idempotent receipt.
- Show a non-blocking reconnect banner and preserve the player's seat.
- Allow a player to refresh the page or switch devices without losing the game.
- Define a host disconnect policy: host is a room role, not a single point of failure. If host leaves, elect the earliest remaining member as lobby host; in-game host identity should not affect rules.
- Use server-side inactivity and turn deadlines; browser close events are unreliable.

7.4 Reactions / emotes

No chat in MVP. Reactions are ephemeral, rate-limited and non-authoritative.

- Provide 8–12 fixed reactions, for example: ■ ■ ■ ■ ■ ❤■ ■ ■.
- Send reaction with roomId, gameId, emoteId and a client-generated nonce; server validates membership and allowlist.
- Limit to e.g. 3 reactions per user per 5 seconds; tune after testing.

- Broadcast with sender display name/avatar snapshot and server timestamp.
- Auto-expire reaction events after 5–10 seconds; do not include them in permanent game archives by default.
- Do not permit arbitrary Unicode text or uploaded images as reactions in MVP.

8. Google login, profiles and friends

8.1 Authentication

- Enable Google provider in Firebase Authentication.
- Use Firebase Auth SDK signInWithPopup on desktop and signInWithRedirect where popup flows are blocked; test Safari/iOS and third-party cookie restrictions.
- Configure authorized domains for localhost, GitHub Pages domain and any custom domain.
- After sign-in, create/update users/{uid} via trusted profile bootstrap or a tightly restricted user-owned profile write.
- Use UID as stable identity; never use display name or email as the primary key.
- Support sign-out and account deletion workflow. Do not promise complete deletion until dependent archives/records are handled according to policy.

8.2 Friend system

For MVP, support friend requests by exact username/short friend code and accept/reject/remove. Avoid open directory search by email. A future feature can add QR or invite links.

Action	Behaviour
Send request	Create pending request if not self, not already friends, not duplicate and recipient permits requests.
Accept	Create friendship record for both users using a canonical pair ID; mark request accepted.
Reject	Mark rejected or delete after a short retention period.
Cancel	Sender may cancel pending request.
Remove friend	Delete friendship relation and update denormalized counts.
Invite to room	Only invite existing friends; invite link/code remains the actual room join credential.
Block (recommended)	Optional but recommended safety control; blocked users cannot send requests or invites.

8.3 Friend ID

Generate a random public friend code that is not the Firebase UID and not derived from email. Index it for exact lookup. Apply rate limiting and generic errors to reduce account enumeration.

8.4 Privacy

- Default profile exposes display name, avatar and friend code only.
- Do not expose email or provider details to other players.
- Allow users to control friend requests and remove friends.
- Provide a report/block pathway before opening public room discovery in a later release.
- Use a neutral fallback avatar if Google photo is unavailable.

9. Security rules and backend trust boundary

9.1 Security principles

- Default deny. Explicitly allow only required reads/writes.

- Clients never write authoritative live game state, balances, property ownership, dice, cards, turn index, winner or archive records.
- Callable functions require Firebase Authentication and validate all payloads with Zod.
- Use App Check for supported web clients as an abuse mitigation layer; it is not a substitute for authorization.
- Enforce membership checks on every room/game read and every command.
- Validate every property, player ID, bid amount, emote ID and trade offer server-side.
- Use rate limits for room creation, join attempts, friend lookup, friend requests and reactions.
- Do not trust client timestamps, display names, room status or expected ownership.
- Use least-privilege service accounts and separate dev/staging/prod Firebase projects.

9.2 Firestore rules sketch

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() {
      return request.auth != null;
    }

    function isRoomMember(roomId) {
      return signedIn() &&
        exists(/databases/$(database)/documents/rooms/$(roomId)/members/$(request.auth.uid));
    }

    match /users/{uid} {
      allow read: if signedIn() && request.auth.uid == uid;
      // Prefer callable function for profile updates.
      allow write: if false;
    }

    match /rooms/{roomId} {
      allow read: if isRoomMember(roomId);
      allow write: if false;

      match /members/{uid} {
        allow read: if isRoomMember(roomId);
        allow write: if false;
      }
    }

    match /games/{gameId} {
      allow read: if signedIn() &&
        resource.data.roomId is string &&
        isRoomMember(resource.data.roomId);
      allow write: if false;

      match /events/{eventId} {
        allow read: if signedIn(); // tighten using parent game membership
        allow write: if false;
      }
    }

    match /gameArchives/{archiveId} {
      allow read: if signedIn() &&
        request.auth.uid in resource.data.participantUids;
      allow write: if false;
    }

    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

This is a starting sketch, not a deploy-ready complete ruleset. The agent must test every rule in the Firebase Emulator Suite and ensure event reads are membership-scoped. Cloud Functions Admin SDK bypasses Firestore Security Rules, so backend authorization checks remain mandatory.

9.3 Callable function contract

Function	Input	Output
createRoom	maxPlayers, mode, timer settings	roomId, inviteCode, inviteUrl
joinRoom	inviteCode	roomId, member summary
setReady	roomId, ready	updated lobby version
startGame	roomId	gameId, initial version
submitGameAction	GameCommand	accepted result, version, event summary
sendReaction	roomId, gameId, emoteId, nonce	acceptedAt
sendFriendRequest	friendCode	requestId
respondFriendRequest	requestId, ACCEPT/REJECT	status
removeFriend	friendUid	success
getMyArchives	cursor, pageSize	archive summaries

10. UI/UX system and accessibility

10.1 Design direction

Minimalistic, calm, legible and game-first. Avoid casino aesthetics, excessive gradients, dense dashboards, tiny text, noisy animations and modal overload. Use a warm off-white canvas, deep ink text, restrained Indian-inspired accent colours and clear colour-group indicators. The board should be visually rich enough to understand ownership but the surrounding chrome should stay quiet.

10.2 Main routes

Route	Purpose
/	Landing / signed-out entry
/home	Dashboard: create, join, friends, recent games
/friends	Friend list, requests, add by friend code
/room/:roomId	Pre-game lobby
/game/:gameId	Live board game
/history	Completed game archives
/profile	Display name, avatar and privacy settings
/settings	Audio, animation, accessibility, sign out

10.3 Lobby layout

- Room code and copy button at the top.
- Player seats as simple cards with avatar, name, host badge and ready state.
- Host controls: max players and allowed game options; do not let host alter settings after start.
- Primary CTA: Ready for players; Start Game for host when eligible.
- Invite Friends button opens a friend picker; copy invite link is always available.
- Show network status unobtrusively.

10.4 Game screen layout

Desktop: central board, compact player strip, right-side action/property panel, bottom event feed and reaction tray. Mobile: board scaled to fit width or horizontally pannable with a clear focus on the current space; player list collapses into a drawer; action panel becomes a bottom sheet. Never require hover to discover a legal action.

10.5 UI states

- Loading: skeleton board and clear loading message; avoid blank screens.
- Waiting: identify whose turn it is and show a subtle countdown if enabled.
- Action required: show one prominent next action and a concise explanation.
- Insufficient cash: explain amount due and available ways to raise funds.
- Offline/reconnecting: banner with retry status; preserve the board view.
- Completed: show winner, standings, rematch/create-room and return-home actions.
- Error: human-readable message, retry when safe, and a correlation ID for support.

10.6 Accessibility

- Meet WCAG 2.2 AA target where practical: contrast, keyboard focus, labels and semantic controls.
- Do not use colour alone to identify property groups; pair colours with names/icons.
- Support reduced motion and mute/sound controls.
- Provide accessible names for dice, emotes, player tokens and board spaces.
- Ensure buttons have touch targets around 44×44 CSS px or better.
- Test zoom to 200%, keyboard-only navigation and screen-reader labels.

10.7 Animation

Keep animation short and interruptible: dice roll, token movement, rent transfer, card reveal and emote pop. The animation is presentation only. The backend result is authoritative and should already be committed. If a listener update arrives while animation is running, queue visual playback but do not delay authoritative state application.

11. Testing, observability and quality gates

11.1 Testing pyramid

Test level	What to test	Tool
Unit	Movement, rent, tax, doubles, jail, auction, mortgage, building, bankruptcy, card effects	Vitest
Property-based	Invariants across random legal action sequences	fast-check
Integration	Cloud Functions + Firestore transactions + auth checks	Firebase Emulator Suite
Component	Dialogs, board panels, forms, player cards, action states	Testing Library
E2E	Two browser contexts join room and complete a game flow	Playwright
Load/soak	Concurrent listeners, repeated commands, reconnects	k6 or controlled scripts
Security	Unauthorized reads/writes, replay, spoofing, rate limits	Emulator tests + manual review

11.2 Game invariants

- Total player cash plus bank-held cash movements reconcile according to the ruleset; no transaction creates or destroys money unless a bank rule says so.
- Each property has at most one owner.

- Each player has at most one active seat per game.
- Only the current player can submit turn actions, except allowed reactions and trade responses.
- Player position is always an integer in [0, 39].
- Development levels are valid and even within each colour group.
- Mortgaged properties generate no rent and cannot have buildings.
- Game version strictly increases on each accepted authoritative transition.
- An accepted commandId is applied at most once.
- Completed games cannot accept new gameplay commands.
- Only participants can read a private archive.

11.3 Must-have automated scenarios

- Create room; join with second user; reject third user if max players is two.
- Duplicate join request does not create duplicate seat.
- Non-host cannot start game; host cannot start before all ready.
- Two clients see the same server-generated dice and resulting board position.
- Out-of-turn roll is rejected.
- Replay of same commandId does not charge twice.
- Two simultaneous bids cannot both win an auction.
- Rent payment with insufficient cash enters debt/liquidation flow.
- Three doubles sends player to Jail and does not resolve third destination.
- Card moves across Start exactly once and awards correct bonus.
- Reconnect after refresh restores same player and game state.
- Emote rate limit is enforced and expired reactions disappear.
- Archive is created and verified before live game cleanup.
- Cleanup retry does not delete archive or fail on already-deleted documents.

11.4 Monitoring

- Structured logs with requestId, roomId, gameId, uid hash/pseudonymous identifier, commandId and error code; avoid logging tokens or private payloads.
- Track callable function latency/error rates, transaction conflicts, failed joins, room abandonment, game completion and cleanup failures.
- Set alerts for repeated function failures, archive failures, unexpected cost spikes and authentication errors.
- Add a simple in-app “Report a problem” flow that captures non-sensitive diagnostics and a correlation ID.

12. CI/CD, hosting and environment setup

12.1 Local development

Use Node.js LTS supported by current Firebase tooling and package manager lockfile. Pin the Node version in .nvmrc or equivalent. Use Firebase Emulator Suite for Auth, Firestore, Functions and Realtime Database wherever supported.

```
# Example setup flow (agent should verify current CLI syntax)
npm create vite@latest indian-board-game -- --template react-ts
cd indian-board-game
npm install
npm install firebase react-router-dom @tanstack/react-query zustand zod
npm install -D vitest @testing-library/react @playwright/test eslint prettier

# Firebase project setup
```

```
npm install -g firebase-tools
firebase login
firebase init
firebase emulators:start
```

12.2 Environment variables

```
# .env.example (public Firebase web config is not a secret,
# but still configure per environment)
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_APP_ID=
VITE_FIREBASE_DATABASE_URL=
VITE_APP_BASE_PATH=/

# Never put service account keys or Admin SDK credentials here.
```

Firebase web configuration is intended to be present in client apps; security comes from Auth, Rules, App Check and backend validation, not hiding the API key. Never commit service-account JSON or private credentials.

12.3 GitHub Pages

- Configure Vite base to the repository path for project pages, or use a custom domain with root base.
- Use HashRouter for robust deep links on GitHub Pages, or configure a tested 404.html SPA fallback.
- Build static assets in GitHub Actions and deploy only the dist output.
- Set repository Pages source to GitHub Actions.
- Use GitHub Environment secrets/variables only for deployment configuration that genuinely requires them.
- Add a custom domain only after DNS and Firebase authorized-domain setup are verified.
- Test invite links, refresh on nested routes, OAuth redirect and mobile Safari on the actual Pages URL.

12.4 CI workflow

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test -- --run
      - run: npm run build
```

The agent must verify supported Node and action versions at implementation time and update the example if current official requirements differ. Pin third-party actions to reviewed versions or commit SHAs for a hardened production workflow.

12.5 Environments

Environment	Purpose	Data
Local	Development with emulators	Synthetic test users
Development	Shared integration branch	Disposable test data
Staging	Pre-release acceptance and mobile tests	Synthetic accounts only
Production	Public live game	Real user accounts; monitored and backed up

Use separate Firebase projects for staging and production. Do not point local development at production by default.

13. Privacy, retention and operational safeguards

13.1 Data minimization

- Collect only UID, display name, avatar URL, friend code, relationship state and gameplay data needed for the service.
- Do not collect phone number, precise location, contacts or address for this game.
- Do not put friend email addresses in room documents.
- Keep reaction events transient; no chat transcript exists.
- Use short-lived invite codes and avoid predictable sequential IDs.

13.2 Retention policy proposal

Data	Suggested retention
Live lobby	Delete after 24 hours of inactivity
Live completed game state	Delete after archive verification and short grace period (e.g. 1 hour)
Transient reactions	5–10 seconds
Presence	Current connection + short last-seen metadata
Game archive	User-visible retention configurable; e.g. 12 months initially
Rejected friend requests	Delete or expire after 30–90 days
Security/operational logs	Minimum period needed for abuse investigation and compliance

Publish a privacy notice before public launch. Explain what game history is retained, who can see it, how long it is kept, how account deletion works, and how users can request deletion. Confirm applicable Indian data-protection requirements with qualified counsel before production.

13.3 Account deletion

Provide a deletion request path. A trusted backend workflow should revoke/delete Firebase Auth identity, remove profile and friend edges, remove pending requests, and handle archives according to the published policy. If archives are retained for other participants, pseudonymize the deleted user's snapshot where legally appropriate. Do not leave orphaned private data.

13.4 Abuse prevention

- Rate limit room creation, code guessing, friend-code lookup and reactions.
- Use App Check and monitor abuse patterns.
- Room codes should have sufficient entropy; use expiry and attempt throttling.
- Validate emote allowlist and payload size.
- Provide block/report features before enabling public discovery.
- Do not let users upload arbitrary assets in MVP.

14. Delivery milestones and coding-agent workflow

The agent should implement sequentially. At the end of each milestone, run checks, summarize changed files, explain decisions and list unresolved risks. Do not start the next milestone if the current one does not build or tests fail.

Milestone	Deliverables	Exit gate
M0 — Product foundation	Requirements, ruleset config, wireframes, architecture decisions, repo scaffold	Approved scope and app boots
M1 — Static UI	Responsive landing, dashboard shell, lobby shell, board prototype using mock data	Responsive visual review

Milestone	Deliverables	Exit gate
M2 — Auth + profiles	Google login, sign-out, profile bootstrap, protected routes	Auth emulator + OAuth smoke test
M3 — Friends	Friend codes, requests, accept/reject/remove, invite picker	Rules and duplicate-request tests
M4 — Rooms	Create/join, code, ready state, host, room expiry	Two browser contexts join same lobby
M5 — Pure game engine	Rules config, state machine, movement, rent, cards, jail, auctions, bankruptcy	Unit/property tests and invariant checks
M6 — Firebase live engine	Callable commands, transactions, versioning, event log, server dice	Concurrent/replay/out-of-turn tests
M7 — Live UX	Board UI, listeners, action panel, event feed, reconnect and presence	Two-device game smoke test
M8 — Reactions	Curated emotes, broadcast, rate limit, expiry	Reaction abuse tests
M9 — Archive + cleanup	Final standings, immutable archive, verified cleanup, history screen	Archive-before-delete integration test
M10 — Hardening	Security rules, App Check, monitoring, performance, accessibility, load test	Security and quality gates pass
M11 — Release	GitHub Actions deploy, custom domain if used, privacy notice, runbooks	Production checklist signed off

14.1 Prompting protocol for the coding agent

Give the agent this manual and the repository. Instruct it to read the entire document before changing files, then produce an implementation plan and questions. Require it to work milestone by milestone, not replace the architecture with a preferred stack without explaining trade-offs.

```
You are the senior full-stack engineer implementing this project.

1. Read the complete implementation manual before coding.
2. First inspect the repository and report its current state.
3. Produce a milestone plan mapped to this manual.
4. Do not skip security, tests, or documentation.
5. Keep all authoritative game logic in pure TypeScript domain modules
   and trusted Firebase Cloud Functions.
6. The browser must never decide dice, money, rent, ownership, turn order,
   auction outcomes, bankruptcy, or winner.
7. Use Firebase Emulator Suite for local integration tests.
8. After each milestone: lint, typecheck, unit tests, build, and summarize.
9. Do not invent Firebase credentials or deploy to production.
10. Ask before changing product scope, data retention, rules or stack.
11. Keep README, .env.example, Firestore rules, indexes and runbooks updated.
12. Never claim a feature works unless tests or a manual verification prove it.
```

14.2 Definition of done for every task

- Implementation matches the requirements and design system.
- Input validation and authorization exist at trust boundaries.
- Loading, error, empty, offline and success states are covered.
- Unit/integration tests cover normal and failure paths.
- TypeScript, lint, tests and production build pass.
- Documentation and environment examples are updated.
- No secrets, personal data dumps or debug logs are committed.
- Changes are small enough to review and revert.

15. Acceptance criteria and launch checklist

15.1 Product acceptance

- User can sign in with Google on desktop and mobile browsers.

- User can create a private room and share a join code/link.
- At least two and up to six authenticated users can join and ready up.
- Host can start only when all required conditions are satisfied.
- All devices show the same server-authoritative state within a reasonable real-time delay.
- Only the correct player can roll or submit turn actions.
- Players can react using curated emotes; no messaging feature is present.
- Users can add friends, accept/reject requests and invite friends.
- Network interruption and refresh do not destroy a player's seat or duplicate actions.
- Completed game archive is visible to participants.
- Live room/game documents and ephemeral state are removed only after archive verification.
- UI is understandable without a tutorial and usable at mobile widths.

15.2 Security acceptance

- Firestore rules deny client writes to live game state and archives.
- Unauthenticated users cannot read private rooms or archives.
- Non-members cannot read room/game snapshots or event history.
- Out-of-turn, stale-version, malformed, replayed and unauthorized commands are rejected.
- Emote and room-creation rate limits work.
- Firebase App Check is configured for production where supported.
- Staging and production are isolated.

15.3 Release checklist

- Custom name and visual identity reviewed for trademark/IP risk.
- Privacy notice and terms are published.
- Firebase billing, budget alerts and quotas are configured.
- Firestore indexes and security rules deployed.
- Functions region matches database region.
- Google OAuth authorized domains and redirect flows tested.
- GitHub Pages base path and deep links tested.
- Two-device and six-player playtests completed.
- Archive/cleanup failure recovery tested.
- Accessibility and reduced-motion checks completed.
- Backup/export and incident runbooks documented.
- Production deploy performed only after explicit approval.

Appendix A — Example board configuration

```
export const BOARD = [
  { id: 0, type: "START", name: "START", reward: 2000 },
  { id: 1, type: "PROPERTY", propertyId: "jaipur_mi_road" },
  { id: 2, type: "CARD", deck: "CIVIC" },
  { id: 3, type: "PROPERTY", propertyId: "jodhpur_clock_tower" },
  { id: 4, type: "TAX", amount: 2000, name: "GST Checkpost" },
  { id: 5, type: "TRANSIT", assetId: "mumbai_local" },
  // Continue through all 40 spaces from the ruleset.
] as const;
```

The agent must encode all 40 spaces and all deed/card definitions in a validated configuration file. Do not use the abbreviated array above as a complete board.

Appendix B — API error contract

```
type ApiErrorCode =
  | "UNAUTHENTICATED"
  | "PERMISSION_DENIED"
  | "ROOM_NOT_FOUND"
  | "ROOM_FULL"
  | "ROOM_NOT_JOINABLE"
  | "ALREADY_IN_ROOM"
  | "NOT_ROOM_MEMBER"
  | "NOT_HOST"
  | "GAME_NOT_STARTED"
  | "GAME_ALREADY_COMPLETED"
  | "NOT_YOUR_TURN"
  | "INVALID_PHASE"
  | "INVALID_ACTION"
  | "INSUFFICIENT_FUNDS"
  | "VERSION_CONFLICT"
  | "RATE_LIMITED"
  | "INTERNAL";

type ApiResult<T> =
  | { ok: true; data: T }
  | { ok: false; error: { code: ApiErrorCode; message: string; retryable: boolean } };
```

Return stable error codes to the UI. Messages should be short and actionable; never expose stack traces, document paths, security-rule details or internal credentials.

Appendix C — Manual test script: two devices

- Device A signs in with Google and creates a room.
- Device A copies the room code.
- Device B signs in with a different Google account and joins using the code.
- Both devices see two players; both mark Ready.
- Host starts. Both devices see the same board, player order and active player.
- Active player rolls. Both devices see identical dice and movement.
- Active player buys an unowned property. Both devices see the owner and cash change.
- Second player lands on that property and pays rent. Both devices see the same result.
- Second player sends an emote. First device sees it disappear after its short display period.
- Refresh Device B. It reconnects to the same seat and current state.
- Complete a game using a test fixture or shortened test ruleset.
- Verify archive exists and is readable by participants.
- Verify live game, room, events, invite lookup and reactions are cleaned up.
- Verify a non-participant cannot read the archive.

Appendix D — Architecture decisions the agent must document

Decision	Required record
Firebase region	Chosen region, latency rationale and product availability check
GitHub Pages routing	Hash routing or SPA fallback and deep-link tests
Reaction transport	RTDB or Firestore, expiry mechanism and rate limit
Invite code	Entropy, normalization, storage/hash lookup and expiry
Archive policy	Fields retained, duration, deletion process and account deletion impact
Turn timeout	Timeout values and exact auto-action behaviour
Host disconnect	Host transfer rules
Game rules version	How config is versioned and old archives interpreted
Cost controls	Expected users/rooms/listeners, budget alerts and quotas

Appendix E — Official documentation to consult

The coding agent must check current official documentation during implementation because Firebase pricing, SDKs, supported runtimes and GitHub Actions evolve. Use official references as the source of truth; do not copy outdated blog tutorials.

- **Firebase Authentication — Google sign-in** — https://firebase.google.com/docs/auth/web/google-signin
- **Cloud Firestore — real-time listeners** — https://firebase.google.com/docs/firestore/query-data/listen
- **Cloud Firestore — transactions and batched writes** — https://firebase.google.com/docs/firestore/manage-data/transactions
- **Cloud Firestore — security rules** — https://firebase.google.com/docs/firestore/security/get-started
- **Firebase Realtime Database — presence** — https://firebase.google.com/docs/database/web/offline-capabilities
- **Cloud Functions for Firebase** — https://firebase.google.com/docs/functions
- **Firebase App Check** — https://firebase.google.com/docs/app-check
- **Firebase Emulator Suite** — https://firebase.google.com/docs/emulator-suite
- **Firestore TTL** — https://firebase.google.com/docs/firestore/ttl
- **GitHub Pages with GitHub Actions** — https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages
- **Vite deployment guide** — https://vite.dev/guide/static-deploy.html
- **React documentation** — https://react.dev/learn

Final instruction to the coding agent

Deliver a secure, maintainable, tested multiplayer web game—not merely a visually convincing board mock-up. The most important engineering outcomes are authoritative game logic, safe concurrent state transitions, reliable reconnect, privacy-aware friends, and archive-before-cleanup lifecycle. Keep the interface minimal, the rules explainable, and the implementation modular. Ship the MVP only when the two-device end-to-end flow and security acceptance criteria pass.

End of manual.